

Clase 023 — Sistemas operativos: procesos, memoria y syscalls

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Tanenbaum & Bos, Modern Operating Systems* 🕒
Duración estimada: **110 min** · Nivel: **Fundamentos**

🎯 Objetivo

Comprender cómo un sistema operativo gestiona procesos, memoria y la frontera entre modo usuario y modo kernel a través de las llamadas al sistema. Este es el sustrato sobre el que ocurren la explotación de memoria, la inyección de código y la evasión, y es imprescindible para las partes de explotación y forense.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** el ciclo de vida y estados de un proceso.
2. **Describir** el layout de memoria de un proceso (stack, heap, code, data).
3. **Diferenciar** modo usuario de modo kernel y el papel de las syscalls.
4. **Rastrear** las llamadas al sistema de un programa.
5. **Relacionar** estos mecanismos con técnicas de ataque y detección.

📖 Temas

#	Tema	Por qué importa
1	Procesos e hilos	Unidad de ejecución
2	Estados y planificación	Cómo comparten la CPU
3	Layout de memoria	Dónde vive cada cosa
4	Memoria virtual	Aislamiento y paginación
5	Modo usuario/kernel	Frontera de privilegios
6	Syscalls	Puerta de entrada al kernel
7	Herramientas de traza	strace, ltrace, procfs
8	Relevancia ofensiva	Inyección, hooking, monitoreo

📖 Definiciones y características

- **Proceso**: programa en ejecución con su propio espacio de memoria. Clave: aislado de otros por memoria virtual.

- **Hilo (thread):** flujo de ejecución dentro de un proceso; comparten memoria. Clave: concurrencia con estado compartido.
- **Stack:** memoria LIFO para llamadas y variables locales. Clave: objetivo de desbordamientos (buffer overflow).
- **Heap:** memoria dinámica gestionada por el programa. Clave: objetivo de vulnerabilidades tipo use-after-free.
- **Syscall:** interfaz para pedir servicios al kernel (open, read, execve). Clave: única vía legítima de un proceso para tocar el hardware/recursos.
- **Modo usuario/kernel:** niveles de privilegio de la CPU (rings). Clave: la separación evita que un proceso comprometa el sistema directamente.

Herramientas y preparación

En Linux/Kali: `ps`, `top` / `htop`, `/proc`, `strace`, `ltrace`, `pmap`, `cat /proc/<pid>/maps`. Necesitas permisos para trazar procesos propios. Un programa C mínimo o cualquier binario servirá de sujeto de estudio. Trabaja en tu VM de laboratorio.

Laboratorio guiado

1. **Explorar un proceso vivo.** Lanza `sleep 1000 &` y examina:

```
bash ps -o pid,ppid,state,cmd -p $! cat /proc/$!/status | head
```

2. **Layout de memoria.** Observa las regiones del proceso:

```
bash pmap $! # o: cat /proc/$!/maps
```

Identifica stack, heap y las bibliotecas mapeadas. 3. **Trazar syscalls.** Mira qué llamadas hace un comando:

```
bash strace -f -e trace=open,openat,read,write ls / 2>&1 | head -30
```

4. **Llamadas a librería** con `ltrace` sobre un binario dinámico:

```
bash ltrace -e 'malloc+free' ./programa 2>&1 | head
```

5. **Modo usuario vs. kernel.** Con `strace -c ls` mide cuánto tiempo pasa el proceso en syscalls vs. en espacio de usuario.
6. **Relevancia ofensiva** (conceptual). Relaciona: un buffer overflow corrompe el **stack**; un `execve` inesperado en `strace` puede delatar una **ejecución maliciosa**; el hooking de syscalls es base de rootkits y de EDR.

 **Nota ética:** traza únicamente procesos propios o en tu laboratorio. Interceptar procesos de otros usuarios sin autorización puede ser ilegal.

Ejercicios

1. Dibuja el layout de memoria de un proceso e indica en qué región vive cada tipo de dato.
2. Explica la diferencia entre proceso e hilo y una implicación de seguridad de cada uno.
3. Con `strace`, identifica todas las syscalls que usa un programa para leer un archivo.

4. Investiga qué es una interrupción/trap y cómo transfiere el control al kernel.
5. Compara `strace` y `ltrace`: ¿qué observa cada uno y para qué sirve en análisis de malware?
6. Explica por qué la separación usuario/kernel es una frontera de seguridad y qué es una "elevación a kernel".

Reto verificable

Analiza un binario desconocido (inofensivo, de tu laboratorio) usando solo herramientas de observación de SO: describe su comportamiento a partir de sus syscalls (archivos que abre, red que usa, procesos que lanza) sin ejecutar nada peligroso fuera del entorno aislado. Entrega un informe con el "perfil de comportamiento" del programa.

Criterio de aceptación: el informe lista las syscalls relevantes observadas con `strace` (E/S de archivos, red, `execve`) y deduce a partir de ellas qué hace el programa, distinguiendo actividad benigna de indicios sospechosos. Reproducible ejecutando el mismo `strace` sobre el binario.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>strace: Operation not permitted</code>	Restricciones de ptrace o proceso ajeno. Traza procesos propios o ajusta <code>ptrace_scope</code> .
<code>ltrace</code> no muestra nada	Binario estático o sin símbolos. Ltrace necesita enlace dinámico.
<code>/proc/<pid>/maps</code> vacío o denegado	Permisos o el proceso terminó. Usa un PID vivo y propio.
Confundir stack con heap	Stack = automático/LIFO; heap = dinámico (malloc). Revisa el layout.
<code>strace</code> ralentiza mucho el programa	Es normal: intercepta cada syscall. Filtra con <code>-e trace=</code> para reducir ruido.

Preguntas frecuentes

? ¿Por qué un pentester necesita entender syscalls? Porque la explotación de memoria, la inyección de shellcode y la evasión operan justo en esta capa. Y en defensa, EDRs y sandboxes se basan en observar syscalls.

? ¿Qué relación hay con los buffer overflows? Un overflow corrompe estructuras en el stack o heap para desviar el flujo de ejecución. Sin entender el layout de memoria, esos ataques (y sus mitigaciones: ASLR, DEP, canarios) no tienen sentido.

? ¿strace sirve para analizar malware? Sí, es una herramienta clave de análisis dinámico: revela qué archivos toca, con qué se conecta y qué procesos lanza, siempre en un entorno aislado.

? ¿Windows tiene equivalentes? Sí: Process Monitor, ETW y API monitors cumplen un papel análogo observando llamadas y actividad del sistema.

Referencias

- Tanenbaum & Bos, *Modern Operating Systems*.
- `man 1 strace` , `man 5 proc`
- Linux syscall reference — <https://man7.org/linux/man-pages/man2/syscalls.2.html>
- Michael Kerrisk, *The Linux Programming Interface*.

Siguiente clase

Clase 024 - Arquitectura de computadores: CPU, registros y memoria